

Linode Setup:

Linode Setup:	1
VM Setup	1
Firewall Configuration:	2

Automated Deployment Using Linode CLI and Cloud Init

Install Linode-CLI

1. Install linode-cli via pip3 or homebrew (a or b):
 - a. pip3 install linode-cli
 - b. brew install linode-cli
2. Next run configure to setup configuration (this is stored in ~/.config/linode-cli)
 - a. linode-cli configure
3. Sample config file for reference:

None

```
[DEFAULT]
default-user = user

[user]
token = <removed>
region = us-ord
type = g6-standard-16
image = linode/ubuntu22.04
suppress-version-warning = true
plugin-obj-access-key = <removed>
plugin-obj-secret-key = <removed>
```

4. Note the token value will be needed for the cloud config below. This will allow the server to authenticate to the bucket and pull down the installer tar file.

Create FW

Next we need to create a FW that will be attached to the linode at the time of creation.

Here is a snippet of the inbound rules. You will need to modify these to list your ipv4 address. The following ports are open to your IP.

- Tcp 22
- Tcp 80
- Tcp 443

- Tcp 8888
- Tcp 8025

None

```
[{"ports": "22", "protocol": "TCP", "addresses": {"ipv4": ["108.91.32.112/32"]}, "action": "ACCEPT", "label": "accept-inbound-SSH", "description": null}, {"action": "ACCEPT", "addresses": {"ipv4": ["108.91.32.112/32"]}, "ports": "80", "protocol": "TCP", "label": "accept-inbound-HTTP", "description": null}, {"action": "ACCEPT", "addresses": {"ipv4": ["108.91.32.112/32"]}, "ports": "443", "protocol": "TCP", "label": "accept-inbound-HTTPS", "description": null}, {"action": "ACCEPT", "addresses": {"ipv4": ["108.91.32.112/32"]}, "description": null, "label": "accept-inbound-HTTP8888", "ports": "8888", "protocol": "TCP"}, {"action": "ACCEPT", "addresses": {"ipv4": ["108.91.32.112/32"]}, "description": null, "label": "accept-inbound-HTTP8025", "ports": "8025", "protocol": "TCP"}]
```

1. Run lincode cli command inserting the updated rules json array from above. **Be sure to capture the ID of the FW this will be used in the linodes create command step 1.**

None

```
linode-cli firewalls create --label my-firewall-test4 --rules.inbound '<insert rules here>' --rules.inbound_policy DROP --rules.outbound_policy ACCEPT --rules.outbound []
```

id	label	status	created	updated
1258794	my-firewall-test4	enabled	2024-11-27T12:50:08	2024-11-27T12:50:08

Create a bucket to store the installer.

1. Create a bucket labeled installer-files
 - a. linode-cli object-storage buckets-create installer-files --cluster us-ord-1
2. Upload the license file and the installation files (tar)
 - a. linode-cli obj put Acme.com-06Feb24.lic installer-files --cluster us-ord-1
 - b. linode-cli obj put noname.online.v3.34.13-lts.tar installer-files --cluster us-ord-1

Run linodes create CLI

1. Modify cloud configuration. See Example below.
2. Replace **<replace with your token>** with the value of the token stored in `~/.config/linode-cli`
3. Make sure to set the root password to a password of your choice.

None

```
#cloud-config
package_update: true
packages:
- python3-pip
write_files:
- content: |
    [DEFAULT]
    default-user = user
    [user]
    token = <replace with your token>
    region = us-ord
    type = g6-dedicated-2
    image = linode/ubuntu22.04
  path: /root/.config/linode-cli
  owner: root:root
runcmd:
- [ mkdir, -p, /opt/noname ]
- [ pip3, install, linode-cli, boto3 ]
- linode-cli obj get installer-files noname.online.v3.34.13-lts.tar --cluster
us-ord-1
- [ tar, -xvf, noname.online.v3.34.13-lts.tar, -C, /opt/noname/ ]
- cd /opt/noname/
- /opt/noname/noname_installer.sh --force --admin-email
"noname-su@nonamesecurity.com" --admin-password "NonameS3curity!"
```

4. Create a new linode with the cli command below.
 - a. Replace **<replace_with_FW_id>** with the ID for the newly created FW.
 - b. Base64encode the cloud configuration above. See example below **Note* I used burp suite, but there are command line options. I won't provide examples here.**
 - c. Replace **<insert metadata here>** with the updated and base64encoded cloud config.

None

```
linode-cli linodes create \
--image 'linode/ubuntu22.04' \
```

```
--private_ip false \
--region us-ord \
--type g6-standard-6 \
--label ubuntu-us-ord \
--root_pass 'AhmC&*2XIUxzP5pj' \
--disk_encryption enabled \
--firewall_id <replace_with_FW_id> \
--metadata.user_data="<insert metadata here>"
```

--metadata.user_data value should be base64encoded string of the cloud-config below

Example:

None

```
--metadata.user_data="I2Nsb3VklWNvbmZpZwpwYWNrYWdlX3VwZGF0ZTogdHJ1ZQpwYWNrYWdlczoK
LSBweXRob24zLXBpcAp3cm10ZV9maWxlczoKLSBjb250ZW500iB8CiAgICBbREVGGVVMVF0KICAgIGRlZm
F1bHQtdXNlciA9IHVzZXIKICAgIFt1c2VyXQogICAgdG9rZW4gPSA8cmVtb3ZlZD4KICAgIHJlZ2lubiA9
IHVzLW9yZAogICAgdHlwZSA9IGc2LWRlZGljYXRlZC0yCiAgICBpbWFnZSA9IGxpbm9kZS91YnVudHUyMi
4wNAogIHBhdGg6IC9yb290Ly5jb25maWcwbGlub2RlLWNsaQogIG93bmVyOiByb290OnJvb3QKcnVuY21k
OgotIFsgbWtkaXIsIC1wLCAvb3B0L25vbmFtZSBdCi0gWyBwaXAzLCBpbmN0YWxsLCA9bW5vZGUtY2xpLC
Bib3RvMyBdCi0gbGlub2RlLWNsaSBvYmogZ2V0IGluc3RhbGxlc1maWxlcyBub25hbWUub25saW51LnYz
LjM0LjEzLWx0cy50YXIsLS1jbHVzdGVyIHVzLW9yZC0xci0gWyB0YXIsIC14dmYsIG5vbmFtZS5vbmxpbm
UudjMuMzQuMTMtbnRzLnRhciwgLUMsIC9vcHQvbm9uYW1lYBdCi0gY2QgL29wdC9ub25hbWUvCi0gL29w
dC9ub25hbWUvbm9uYW1lX2luc3RhbGxlc15zaCAtLWZvcnNlIC0tYWRtaW4tZW1haWwgIm5vbmFtZS1zdU
Bub25hbWVzZW51cm10eS5jb20iIC0tYWRtaW4tZGFzc3dvcnQgIk5vbmFtZVMzY3VyaXR5ISI="
```

*note if using a different region the cluster value will be different.

This cloud config can also be used in the cloud manager UI. When creating a new linode you can insert this into the user data section. [Read More Here](#)

Add User Data ⓘ

User Data ⓘ

runcloud:

- [mkdir, -p, /opt/noname]

- [pip3, install, linode-cli, boto3]

- linode-cli obj get installer-files noname.online.v3.34.13-lts.tar --cluster us-ord-1

- [tar, -xvf, noname.online.v3.34.13-lts.tar, -C, /opt/noname/]

- cd /opt/noname/

- /opt/noname/noname_installer.sh --force --admin-email "noname-su@nonamesecurity.com" --admin-password "NonameS3curity!"

5. Run command **linode-cli linodes ls** to get status of linode deployment.

id	label	region	type	image	status	ipv4	disk_encryption	placement_group.label
67637222	ubuntu-us-ord	us-ord	g6-standard-6	linode/ubuntu22.04	provisioning	172.234.212.147	enabled	

```
chadb@sjc-mps2 ~ % linode-cli linodes ls
```

id	label	region	type	image	status	ipv4	disk_encryption	placement_group.label
60782693	ubuntu-us-central	us-central	g6-standard-6	linode/ubuntu22.04	running	72.14.178.150	disabled	
67637222	ubuntu-us-ord	us-ord	g6-standard-6	linode/ubuntu22.04	provisioning	172.234.212.147	enabled	

```
chadb@sjc-mps2 ~ % linode-cli linodes ls
```

id	label	region	type	image	status	ipv4	disk_encryption	placement_group.label
60782693	ubuntu-us-central	us-central	g6-standard-6	linode/ubuntu22.04	running	72.14.178.150	disabled	
67637222	ubuntu-us-ord	us-ord	g6-standard-6	linode/ubuntu22.04	running	172.234.212.147	enabled	

6. (Optional) Once status show running
- ssh root@<ipv4address>
 - Root password will be the value used in `--root_pass`.
7. (Optional) Monitor the cloud init status with the following commands
- tail -f /var/log/cloud-init-output.log. Example message showing install finished.

None

```
2024-11-27 02:14:57 - Installation - [INFO]: *** Installation Ended Successfully ***
```

- tail -f /var/log/cloud-init.log
- docker service ls

Delete Linode Using CLI

- Run `linode-cli linodes ls`
 - Capture the ID of the linode you want to delete.
- Run `linode-cli linodes delete <id>`

Manual Deployment

- Make sure to select Ubuntu 22.04 LTS. **24.04 LTS not supported on installer.**
- LInode Plan I used 16 GB but the 8GB plan should work

Linodes / Create

Getting Started

DistributionsMarketplaceStackScriptsImagesBackupsClone Linode

Choose an OS

Images

Ubuntu 22.04 LTS

Region

How Data Center Pricing Works

You can use [our speedtest page](#) to find the best region for your current location.

Region

Dallas, TX (us-central)

Linode Plan

Choosing a Plan

Dedicated CPUShared CPUHigh MemoryGPUPremium CPU

Shared CPU instances are good for medium-duty workloads and are a good mix of performance, resources, and price. [Learn more](#) about our Shared CPU plans.

Plan	Monthly	Hourly	RAM	CPUs	Storage	Transfer	Network In / Out
☐ Nanode 1 GB	\$5	\$0.0075	1 GB	1	25 GB	1 TB	40 Gbps / 1 Gbps
☐ Linode 2 GB	\$12	\$0.018	2 GB	1	50 GB	2 TB	40 Gbps / 2 Gbps
☐ Linode 4 GB	\$24	\$0.036	4 GB	2	80 GB	4 TB	40 Gbps / 4 Gbps
☐ Linode 8 GB	\$48	\$0.072	8 GB	4	160 GB	5 TB	40 Gbps / 5 Gbps
☒ Linode 16 GB	\$96	\$0.144	16 GB	6	320 GB	8 TB	40 Gbps / 6 Gbps

Firewall Configuration:

The screenshot shows the Linode Firewall configuration page for 'MainFW'. The 'Rules' tab is active, showing 'Inbound Rules' and 'Outbound Rules' sections. The 'Inbound Rules' table lists three rules: 'accept-inbound-SSH' (TCP, port 22), 'accept-inbound-HTTP' (TCP, port 80), and 'accept-inbound-HTTPS' (TCP, port 443). All rules have a source of '0.0.0.0/0.0.0.0' and an action of 'Accept'. The 'Outbound Rules' section is empty, showing a message 'No outbound rules have been added.' and a default policy of 'Accept'. The interface includes buttons for 'Add An Inbound Rule', 'Add An Outbound Rule', 'Discard Changes', and 'Save Changes'.

Firewalls / MainFW [Docs](#)

[Rules](#) [Linodes \(1\)](#) [NodeBalancers \(0\)](#)

Inbound Rules

[Add An Inbound Rule](#)| Label | Protocol | Port Range | Sources | Action | |
| --- | --- | --- | --- | --- | --- |
| accept-inbound-SSH | TCP | 22 | 0.0.0.0/0.0.0.0 | Accept | [Edit](#) [Clone](#) [Delete](#) |
| accept-inbound-HTTP | TCP | 80 | 0.0.0.0/0.0.0.0 | Accept | [Edit](#) [Clone](#) [Delete](#) |
| accept-inbound-HTTPS | TCP | 443 | 0.0.0.0/0.0.0.0 | Accept | [Edit](#) [Clone](#) [Delete](#) |

Default inbound policy: This policy applies to any traffic not covered by the inbound rules listed above. [Drop](#)

Outbound Rules

[Add An Outbound Rule](#)| Label | Protocol | Port Range | Destinations | Action |
| --- | --- | --- | --- | --- |
| No outbound rules have been added. | | | | |

Default outbound policy: This policy applies to any traffic not covered by the outbound rules listed above. [Accept](#)

[Discard Changes](#) [Save Changes](#)

Noname Setup:

- Ssh to vm
Command:
`ssh root@<linode vm IP>`
- Copy over tar file:
Command:
`scp /Users/chadb/Downloads/3.32.5.tar root@<linode vm IP>:~/`
- Create Noname folder
Commands:
`mkdir -p /opt/noname`
`cd /opt/noname`
- Run installer
Command:
`./noname_installer.sh --force --admin-email 'noname-su@nonamesecurity.com' --admin-password 'NonameS3curity!'`
- Successful install should look like this:

```

2024-07-01 15:05:12 - Deployment - [INFO]: Deploy docker swarm stack
Creating service noname_active_lab
Updating service noname_antigen (id: 146ghpk72e41x3l9q3bwlboxq)
Updating service noname_dal (id: lz6kn1i57nlqvdeh1a6jc454)
Updating service noname_hasura (id: upfhpsfrv8mh68euxywljz19)
Updating service noname_integrations (id: sx0p9rj19xwwsgf92ju1hpt1j)
Updating service noname_nginx (id: 2bl8gdqcuphqv48ka04194vab)
Creating service noname_rabbitmq
Updating service noname_v3-integrations (id: kawnhps7hm9pc845pcvbtrxkl)
Updating service noname_backend (id: ngtyji10czu4sdp5bid0nmq4m)
Updating service noname_postgres (id: ix24jv8wvtosf2tfn6bw4j0)
Updating service noname_router (id: vpwyh43siqhcljydy78lrtr6a)
Creating service noname_active_backend
Creating service noname_active_frontend
Creating service noname_active_hasura
Updating service noname_engine (id: mdtguu7x18jkwxpulrmcqj6ig)
Updating service noname_integrations-adapter (id: o4eh74a079c6h2p4iwdvdrkep)
Updating service noname_nats (id: tuh8xmx75nkw4hdlsilhwih)
Updating service noname_neo4j (id: 6wzigxhu006vkm9vcvpyhsnwd)
Updating service noname_noauth (id: gmnpe24w4swlecl6c87s34bm)
Updating service noname_noqueue (id: xuldluzc9dck7awfid4irurwu)
Updating service noname_syslog-ng (id: 1qeinpemboi1ush78vftjrb2f)
Creating service noname_active_worker
Updating service noname_nomanage (id: 9vcxj1ddchaqyjmt2s6lpfqc9)
Updating service noname_platform (id: ihb1dkrpwp29773egpa1mdsup)
2024-07-01 15:06:03 - Deployment - [INFO]: Waiting for docker swarm
2024-07-01 15:13:59 - Installation - [INFO]: Creating a symlink to noname command
2024-07-01 15:13:59 - Installation - [WARN]: Skipping Activation post-deployment.
2024-07-01 15:13:59 - Installation - [INFO]: *** Installation Ended Successfully ***

```

-

Install Crapi

- Install docker-compose
 - Command:


```
apt install docker-compose
```
- Install crapi
 - Commands:


```
curl -o docker-compose.yml
https://raw.githubusercontent.com/OWASP/crAPI/main/deploy/
docker/docker-compose.yml

docker-compose pull

docker-compose -f docker-compose.yml --compatibility up -d
```
- Stop Crapi
 - Command:


```
docker-compose -f docker-compose.yml --compatibility down
--volumes
```

- **Modify Ports**

Commands:

```
vim docker-compose.yml
```

Modify this line:

```
- "${LISTEN_IP:-127.0.0.1}:8888:80"
```

To:

```
- "8888:80"
```

- Repeat steps above but for mailhog ports

Modify this line:

```
- "${LISTEN_IP:-127.0.0.1}:8025:8025"
```

To:

```
- "8025:8025"
```

- Start Docker and test connectivity to Linode IP on ports 8888 and 8025

- `docker-compose -f docker-compose.yml --compatibility up -d`

- Open chrome:

- Try CRAPI

- `http://<VM IP>:8888/login`

- Try MailHog

- `http://<VM IP>:8025`

Crapi Baseline Script

Setup Script:

- Make sure node is install on the machine you will run the baseline from
- Extract baseline script
- Edit config.js; set the key crapi and mailhog to the respective URLs for your crapi and mailhog instances. If you're not running HTTPS just change the protocol to HTTP and the script will still work
- CD into baseline script folder and run:
 - `yarn install` (preferred)
 - or if you're using NPM `npm install`

Run the baseline:

- `USERS_TO_SIMULATE=3000 node . || true`

- If you want to include the Jane Doe and John Smith accounts used for Active Testing portion of the workshop, set FIRST_RUN=true. EG:
- FIRST_RUN=true USERS_TO_SIMULATE node . || true

From here you can run the baseline whenever you want to drive additional traffic as it will keep generating random user info